

Tema 06 — Team Pokémon

Tema 06 — Team Pokémon

Obiettivo

Realizzare un sito per comporre un team Pokémon a partire da un tipo specifico.

API remota da usare

API: **PokéAPI**

Documentazione ufficiale: <https://pokeapi.co/docs/v2>

Endpoint consigliati:

- `https://pokeapi.co/api/v2/type/fire`

Campi principali da usare

Campo	Significato
<code>pokemon</code>	Lista Pokémon del tipo
<code>name</code>	Nome
<code>sprites.front_default</code>	Immagine
<code>height</code>	Altezza
<code>weight</code>	Peso

Pagine consigliate

Crea almeno 3 pagine HTML. Puoi usare questi nomi:

- `index.html`
- `team.html`
- `dettagli.html`

Cosa deve mostrare il sito

- Mostrare Pokémon di tipo fuoco.
- Spiegare come cambiare tipo Pokémon nell endpoint.

Struttura minima richiesta

La repository deve contenere almeno:

```
nome-progetto/
├── README.md
├── index.html
├── pagina-1.html
├── pagina-2.html
├── style.css
├── script.js
├── data.json
├── docs/
│   ├── installazione.md
│   ├── faq.md
│   └── api.md
├── assets/
└── immagini/
```

Il file `data.json` serve come dati di fallback se la API remota non risponde. Il sito deve provare prima a usare la API remota e poi, in caso di errore, usare i dati locali.

Elementi HTML richiesti per lo script

Nella pagina in cui vuoi mostrare i dati dinamici inserisci almeno:

```
<div id="status-message"></div>
<div id="data-container" class="row g-4"></div>
```

Collega lo script prima della chiusura di `body`:

```
<script src="script.js"></script>
```

Requisiti tecnici

- Sito multipagina con almeno 3 pagine HTML.
- Navbar funzionante e coerente in tutte le pagine.
- Uso di Bootstrap per layout, card, bottoni, alert o tabelle.
- File `style.css` personalizzato.
- Dati caricati da API remota tramite JavaScript.
- Fallback da `data.json` se la API non risponde.
- Messaggio di errore o avviso leggibile usando un alert Bootstrap.
- Percorsi relativi corretti per CSS, JS, immagini e documenti.

Documentazione obbligatoria

Il progetto deve contenere:

- `README.md`: descrizione del progetto, tecnologie usate, requisiti necessari, struttura della repository, link ai documenti.
- `docs/installazione.md`: come clonare/aprire il progetto, avviare il server locale e aprire la pagina corretta.
- `docs/faq.md`: almeno 5 domande frequenti con risposte utili.
- `docs/api.md`: nome API, endpoint usati, campi principali, dove vengono mostrati nel sito, fallback locale.

Bonus possibili

- +5 se il sito è bello, curato e coerente graficamente.
- +5 se GitHub Pages funziona ed è linkato nel README.
- +5 se è presente una licenza e viene spiegato perché è stata scelta.
- +5 se la documentazione contiene screenshot e/o GIF utili.

Checklist finale

☐ La navbar collega tutte le pagine.

- ☐ Bootstrap è caricato correttamente.
- ☐ `style.css` è collegato.
- ☐ `script.js` è collegato.
- ☐ I dati della API vengono mostrati nella pagina.
- ☐ Se la API non risponde, vengono mostrati i dati di `data.json`.
- ☐ Il README linka `docs/installazione.md`, `docs/faq.md` e `docs/api.md`.
- ☐ I percorsi relativi funzionano.